Hospital Management System (HMS) - Project Report

1. Student Details

- **Name:** S SHRIRAM
- **Roll Number:** 24F3004661
- **Email:** 24f3004661@ds.study.iitm.ac.in
- **About Me:** I am a student in the IIT Madras BS Degree program with a keen interest in backend development and building scalable web applications using Python and Flask.

2. Project Details

- **Project Title:** Hospital Management System (HMS)
- **Problem Statement:**

  Hospitals often struggle with manual scheduling, leading to double-booking and inefficient patient history tracking. The goal was to build a digital system that streamlines doctor-patient interactions, manages appointments efficiently, and maintains secure medical records.

- **Approach:**

  I developed a web application using the Flask framework. The architecture follows a Modular Blueprint pattern to separate concerns (Admin, Doctor, Patient). I implemented a custom Availability Grid algorithm to handle scheduling and prevent double-booking. The database is initialized programmatically using SQLite.

3. AI/LLM Declaration

I used AI tools (Large Language Models) to assist with:

- Debugging complex Jinja2 template syntax errors.
- Generating the initial boilerplate code for the Flask Blueprints.
- Refining the CSS styling for a cleaner user interface.
- *Extent of Use:* Approximately 30% of the code was AI-assisted, particularly for frontend styling and boilerplate generation. All core logic (scheduling, database queries, and route protection) was verified and integrated manually.

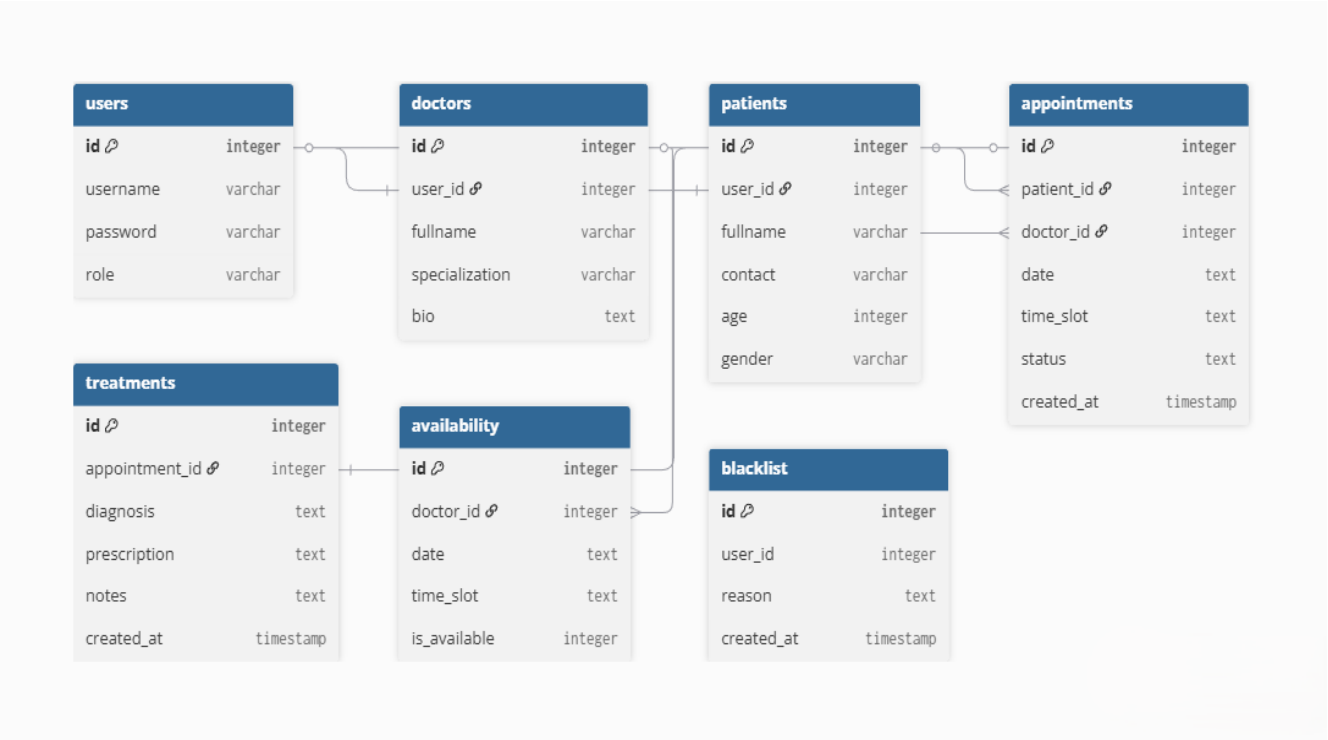
4. Technologies and Frameworks Used

Technology	Purpose
Flask	Core backend web framework for routing and logic.
SQLite	Native database for storing users, appointments, and history.
Jinja2	Template engine for rendering dynamic HTML pages.
Werkzeug	Security library used for password hashing and verification.
HTML/CSS	Custom frontend styling (No external Bootstrap library used).

5. Database Schema Design

The application uses a relational SQLite database with the following tables:

- **Users:** Stores login credentials (username, password_hash) and role (Admin/Doctor/Patient).
- **Doctors:** Linked to Users table via Foreign Key. Stores profile details (specialization, bio).
- **Patients:** Linked to Users table. Stores personal info (age, contact).
- **Appointments:** The central table connecting Doctor and Patient. Stores date, time_slot, and status (Booked/Cancelled/Completed).
- **Availability:** Manages the doctor's schedule. Each row represents a specific open time slot for a specific date.
- **Treatments:** Stores medical history (diagnosis, prescription) linked to a completed Appointment.
- **Blacklist:** Stores IDs of banned users to prevent login access.



6. API Design

I implemented an API endpoint to allow external retrieval of doctor data. The definition is provided in the api.yaml file submitted in the zip.

Endpoint	Method	Description
/patient/api/doctors	GET	Returns a JSON list of all registered doctors and their specializations.

7. Architecture and Features

Architecture

The project is organized into **Flask Blueprints** for scalability:

- auth.py: Handles Login, Registration, and Session management.
- admin.py: Manages high-level CRUD operations for doctors and patients.
- doctor.py: Handles appointment completion and availability setting.
- patient.py: Handles booking logic and search functionality.

Key Features Implemented

1. **Dynamic Availability Grid:** Doctors can create specific time slots for the next 7 days. Patients see a visual grid where they can click to book instant slots.
2. **Split-View Dashboard:** The Admin dashboard automatically separates "Upcoming" and "Past" appointments for better organization.
3. **Smart Search:**
    - **Admins** can search Patients by Name or ID.
    - **Patients** can search Doctors by Name or Department.
4. **Security:**
    - Password hashing for all users.
    - Role-based decorators (@admin_required) protect sensitive routes.
    - Blacklisting system that not only bans users but automatically cancels their future appointments.

8. Video Presentation

- **Link:**

  https://drive.google.com/file/d/1HxFz6ugh2CjvFb-4vZQ-T4-XldN0HhT6/view?usp=sharing